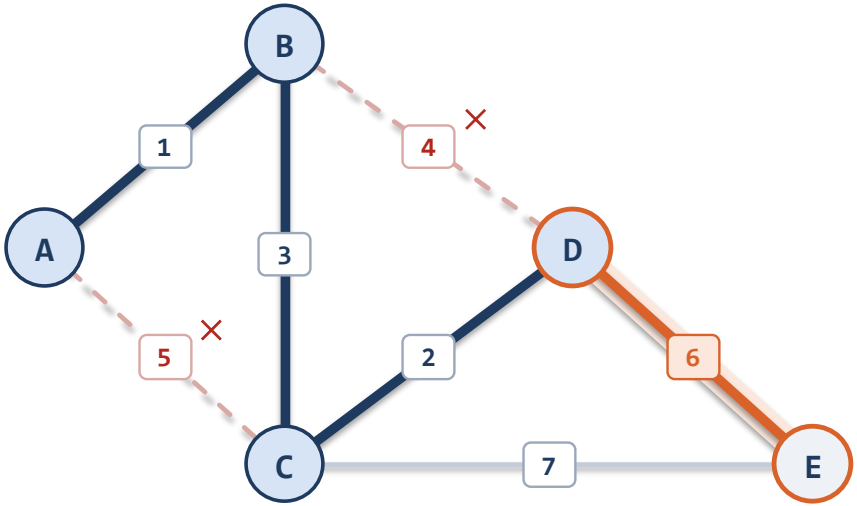
CS3212 Algorithms

Aya Zirikly
11/10/2026

Kruskal trace, step 6 of 6: edge (D, E), w = 6

Graph G, tree edges T in bold



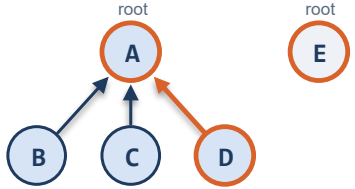
Sorted edges

(A, B)	1	✓ in T
(C, D)	2	✓ in T
(B, C)	3	✓ in T
(B, D)	4	✕ cycle
(A, C)	5	✕ cycle
(D, E)	6	→ now
(C, E)	7	

Union-find state

	A	B	C	D	E
parent	A	A	A	A	E
rank	2	0	1	0	0

Forest (arrows point to parent)



`T = {(A,B), (C,D), (B,C)}` `w(T) = 6` `|T| = 3 / 4`

Not quite. FIND(D) = A and FIND(E) = E: the roots differ.

Different roots mean (D, E) joins two separate trees, so it cannot close a cycle.

Try again | Show answer

Question 1 of 4

If w(B, D) were 2.5 instead of 4, which edge would Kruskal reject first?

- (B, C)
- (B, D)
- (A, C)
- none

Click an answer.

Question 1 of 4

If w(B, D) were 2.5 instead of 4, which edge would Kruskal reject first?

- (B, C)
- (B, D)
- (A, C)
- none

✓ Correct

Order becomes AB 1, CD 2, BD 2.5, BC 3.

(B, D) now joins {A, B} and {C, D}, so (B, C) closes the cycle B – D – C.

MST weight drops to 1 + 2 + 2.5 + 6 = 11.5.

Next question →

Question 2 of 4

Why does Kruskal never look at (C, E): 7?

- `|T| = |V| − 1`
- `largest weight`
- `C, E separate`

Click an answer.

Question 2 of 4

Why does Kruskal never look at (C, E): 7?

`|T| = |V| − 1` | `largest weight` | `C, E separate`

✓ Correct

After (D, E) the tree has 4 = |V| − 1 edges, so line 9 breaks.

Had it run on, FIND(C) = FIND(E) = A and (C, E) would be rejected.

Next question →

Question 3 of 4

After step 4, what is parent[D]?

D | C | A

Click an answer.

Question 3 of 4

After step 4, what is parent[D]?

D | C | A

✓ Correct

FIND(D) walked D → C → A.

Path compression then set parent[D] = A, so step 6 finds D's root in one hop.

Next question →

Question 4 of 4

Could a different tie-breaking rule give a different MST here?

yes

no

Click an answer.

Question 4 of 4

Could a different tie-breaking rule give a different MST here?

yes

no

✓ Correct

All seven weights are distinct, so the MST is unique.

Tie-breaking only matters when equal weights compete.

Continue →

Prim's Algorithm Mechanics & Priority Queue Engine

From Kruskal to Prim

- **Kruskal's Approach (Edge-Centric):** Maintains a global forest of disjoint trees, merging components by selecting the globally cheapest safe edge anywhere in the graph. Uses **Disjoint Set Union (DSU)**.
- **Prim's Approach (Vertex-Centric):** Starts from an initial root vertex s and grows a **single continuous tree** outward like an ink spot. Uses a **Min-Priority Queue**.

The Cut Property in Prim's Algorithm

- **Set S:** The set of vertices already included in the growing Minimum Spanning Tree.
- **Set V - S:** The set of vertices not yet in the tree.
- **Greedy Choice:** Examine all edges crossing the cut `(S, V - S)`. The Cut Property guarantees that the **lightest crossing edge** `(u, v)` (where `u` is in `S` and `v` is in `V - S`) is safe to add to the MST.

Core Data Structures & Maintenance

To quickly find the lightest crossing edge without scanning all edges every turn:

- dist[v] (or key[v]): Stores the minimum edge weight connecting non-tree vertex v to *any* vertex currently in S.
- parent[v]: Stores the vertex in S that yields the minimum edge weight to v.
- Min-Priority Queue (Q): Holds all non-tree vertices V - S, keyed by their dist[v] value.

```
ALGORITHM Prim(G = (V, E), w, s):
    FOR EACH vertex u in V:
        dist[u] = Infinity
        parent[u] = NULL
        in_tree[u] = FALSE

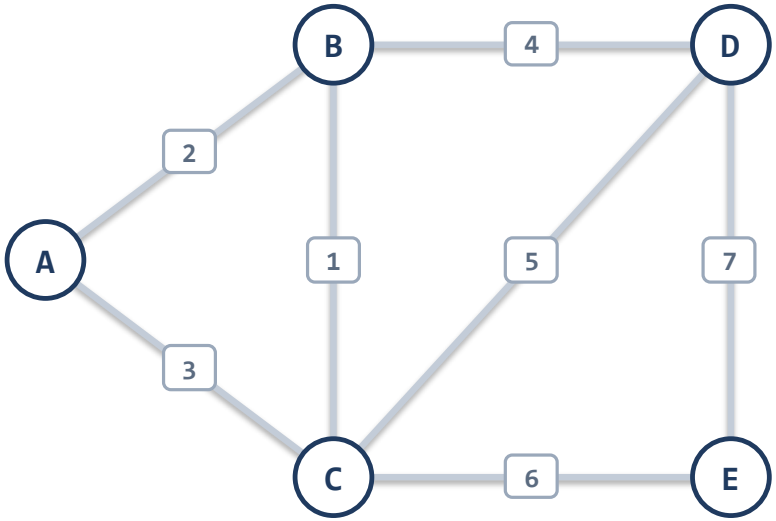
    dist[s] = 0
    Build Min-Priority Queue Q containing all vertices in V keyed by dist[]

    WHILE Q is not empty:
        u = Extract-Min(Q)
        in_tree[u] = TRUE

        FOR EACH neighbor v of u:
            IF in_tree[v] == FALSE AND w(u, v) < dist[v]:
                dist[v] = w(u, v)
                parent[v] = u
                Decrease-Key(Q, v, dist[v])
```

Prim trace: initial state

S shaded · crossing edges dashed · T in bold



T = ∅ w(T) = 0 |T| = 0 / 4

Edges vs. the cut

(B, C)	1
(A, B)	2
(A, C)	3
(B, D)	4
(C, D)	5
(C, E)	6
(D, E)	7

sorted by weight

Priority queue Q

v	key	π	
A	0	–	in Q
B	∞	–	in Q
C	∞	–	in Q
D	∞	–	in Q
E	∞	–	in Q

Q = {A:0, B:∞, C:∞, D:∞, E:∞}

key[v] = lightest edge from v into S

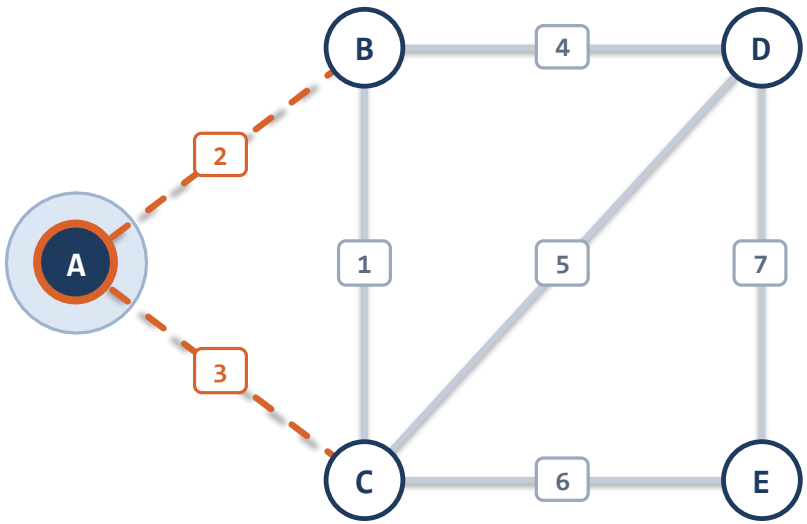
Initialize: key[A] = 0, every other key = ∞, π = NIL, and Q holds all five vertices.

S (the tree side of the cut) is empty. Nothing is in T yet.

Click → to run EXTRACT-MIN. It returns A, because key[A] = 0 is the smallest key.

Prim trace: start at A

S shaded · crossing edges dashed · T in bold



Edges vs. the cut

(B, C)	1	
(A, B)	2	crosses
(A, C)	3	crosses
(B, D)	4	
(C, D)	5	
(C, E)	6	
(D, E)	7	

sorted by weight

Priority queue Q

v	key	π	
A	0	–	in S
B	2	A	in Q
C	3	A	in Q
D	∞	–	in Q
E	∞	–	in Q

`Q = {B:2, C:3, D:∞, E:∞}`

key[v] = lightest edge from v into S

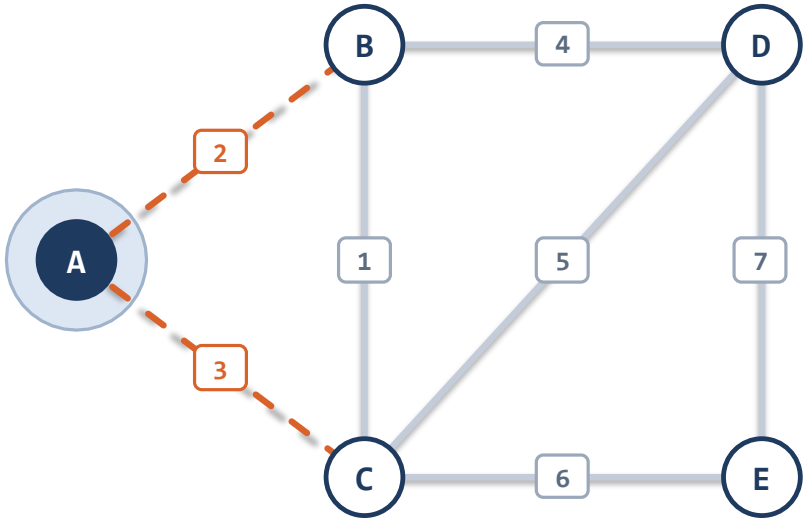
`T = ∅` `w(T) = 0` `|T| = 0 / 4`

EXTRACT-MIN returns A. Now S = {A}, and the cut is ({A}, {B, C, D, E}).

Scan A's neighbours and lower their keys: key[B] ← 2, π[B] ← A, key[C] ← 3, π[C] ← A.
The keys in Q are now exactly the crossing edges into each vertex.

Prim trace, step 1 of 4: which edge does Prim add?

S shaded · crossing edges dashed · T in bold



T = ∅ w(T) = 0 |T| = 0 / 4

Edges vs. the cut

(B, C)	1	
(A, B)	2	crosses
(A, C)	3	crosses
(B, D)	4	
(C, D)	5	
(C, E)	6	
(D, E)	7	

sorted by weight

Priority queue Q

v	key	π	
A	0	–	in S
B	2	A	in Q
C	3	A	in Q
D	∞	–	in Q
E	∞	–	in Q

Q = {B:2, C:3, D:∞, E:∞}

key[v] = lightest edge from v into S

Cut (S, V − S) with S = {A}. Which edge does Prim add next?

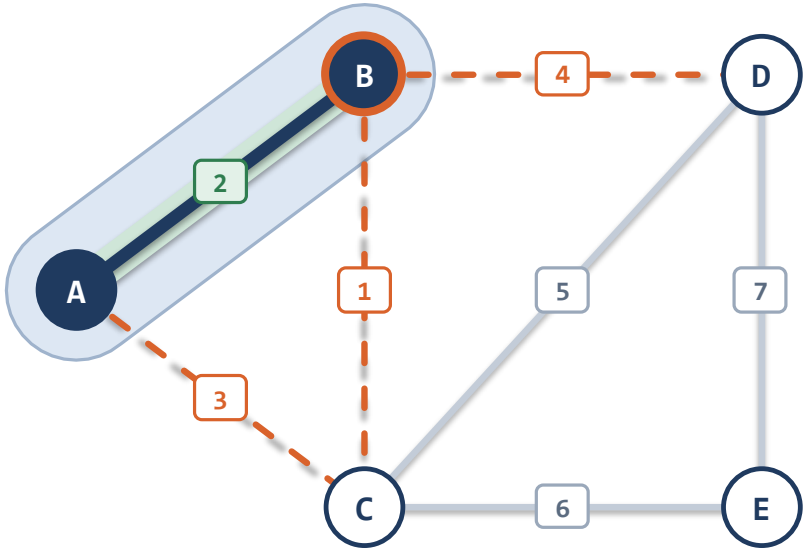
Crossing edges: (A, B) 2, (A, C) 3.

Vote by clicking an edge on the right.

(A, B) 2 (A, C) 3

Prim trace, step 1 of 4: add (A, B), w = 2

S shaded · crossing edges dashed · T in bold



T = {(A,B)} w(T) = 2 |T| = 1 / 4

Edges vs. the cut

(B, C)	1	crosses
(A, B)	2	✓ added
(A, C)	3	crosses
(B, D)	4	crosses
(C, D)	5	
(C, E)	6	
(D, E)	7	

sorted by weight

Priority queue Q

v	key	π	
A	0	–	in S
B	2	A	in S
C	1	B	in Q
D	4	B	in Q
E	∞	–	in Q

Q = {C:1, D:4, E:∞}

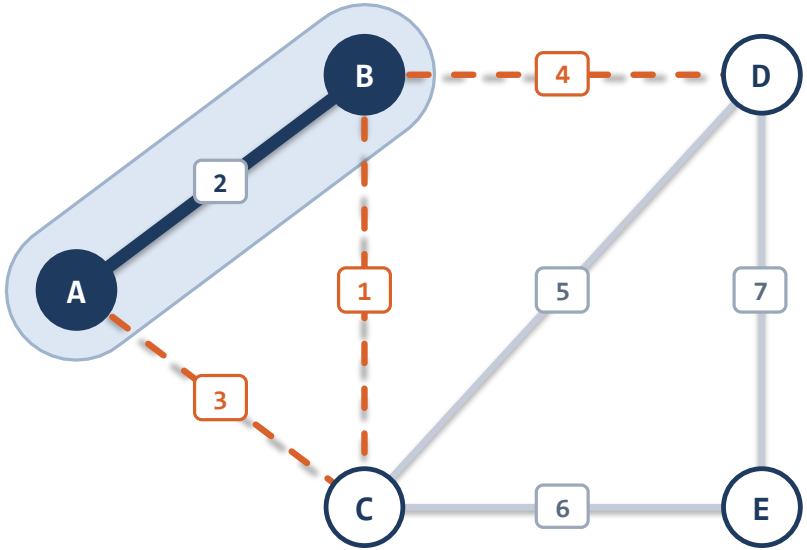
key[v] = lightest edge from v into S

EXTRACT-MIN returns B (key 2). Add (A, B) to T. w(T) = 2.

Update neighbours still in Q: key[C] ← 1 (was 3), π[C] ← B; key[D] ← 4, π[D] ← B.
Cut property: (A, B) is the lightest edge crossing the cut, so it belongs to some MST.

Prim trace, step 2 of 4: which edge does Prim add?

S shaded · crossing edges dashed · T in bold



T = {(A,B)} w(T) = 2 |T| = 1 / 4

Edges vs. the cut

(B, C)	1	crosses
(A, B)	2	✓ in T
(A, C)	3	crosses
(B, D)	4	crosses
(C, D)	5	
(C, E)	6	
(D, E)	7	

sorted by weight

Priority queue Q

v	key	π	
A	0	–	in S
B	2	A	in S
C	1	B	in Q
D	4	B	in Q
E	∞	–	in Q

Q = {C:1, D:4, E:∞}

key[v] = lightest edge from v into S

Cut (S, V − S) with S = {A, B}. Which edge does Prim add next?

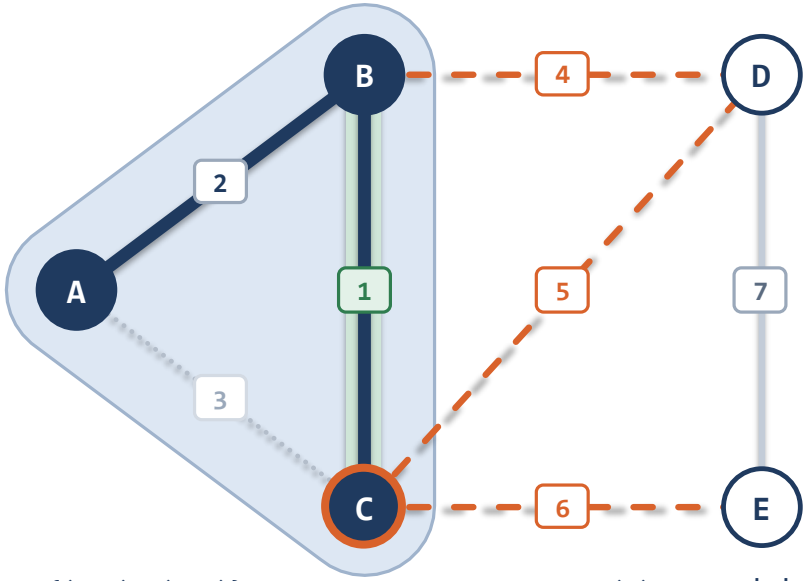
Crossing edges: (B, C) 1, (A, C) 3, (B, D) 4.

Vote by clicking an edge on the right.

(B, C) 1 | (A, C) 3 | (B, D) 4

Prim trace, step 2 of 4: add (B, C), w = 1

S shaded · crossing edges dashed · T in bold



T = {(A,B), (B,C)} w(T) = 3 |T| = 2 / 4

Edges vs. the cut

(B, C)	1	✓ added
(A, B)	2	✓ in T
(A, C)	3	inside S
(B, D)	4	crosses
(C, D)	5	crosses
(C, E)	6	crosses
(D, E)	7	

sorted by weight

Priority queue Q

v	key	π	
A	0	–	in S
B	2	A	in S
C	1	B	in S
D	4	B	in Q
E	6	C	in Q

Q = {D:4, E:6}

key[v] = lightest edge from v into S

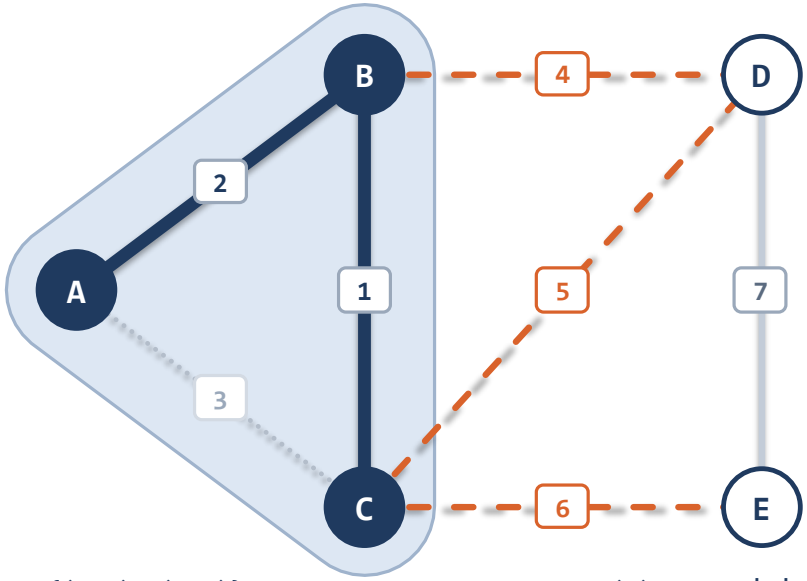
EXTRACT-MIN returns C (key 1). Add (B, C) to T. w(T) = 3.

Update neighbours still in Q: key[D] stays 4, since w(C, D) = 5 is not smaller; key[E] ← 6, π[E] ← C.

Cut property: (B, C) is the lightest edge crossing the cut, so it belongs to some MST.

Prim trace, step 3 of 4: which edge does Prim add?

S shaded · crossing edges dashed · T in bold



T = {(A,B), (B,C)} w(T) = 3 |T| = 2 / 4

Edges vs. the cut

(B, C)	1	✓ in T
(A, B)	2	✓ in T
(A, C)	3	inside S
(B, D)	4	crosses
(C, D)	5	crosses
(C, E)	6	crosses
(D, E)	7	

sorted by weight

Priority queue Q

v	key	π	
A	0	–	in S
B	2	A	in S
C	1	B	in S
D	4	B	in Q
E	6	C	in Q

Q = {D:4, E:6}

key[v] = lightest edge from v into S

Cut (S, V − S) with S = {A, B, C}. Which edge does Prim add next?

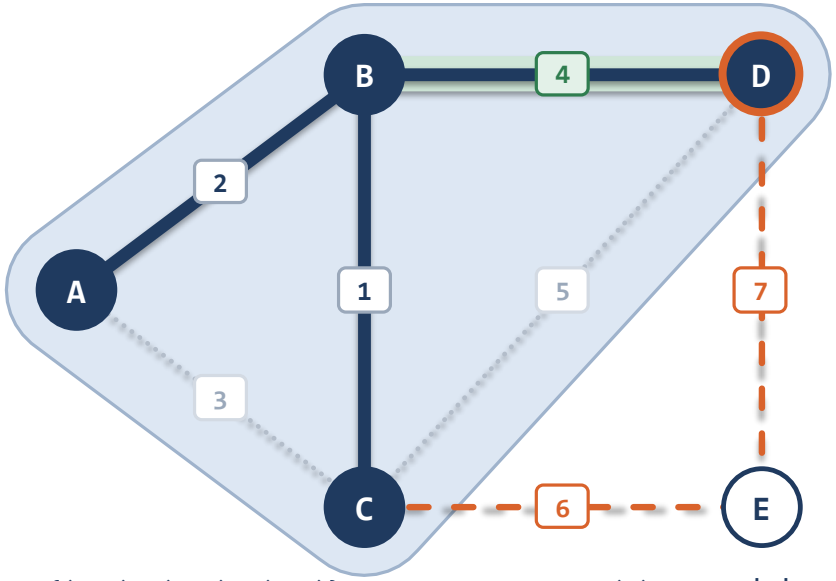
Crossing edges: (B, D) 4, (C, D) 5, (C, E) 6.

Vote by clicking an edge on the right.

(A, C) 3 | (B, D) 4 | (C, D) 5 | (C, E) 6

Prim trace, step 3 of 4: add (B, D), w = 4

S shaded · crossing edges dashed · T in bold



T = {(A,B), (B,C), (B,D)} w(T) = 7 |T| = 3 / 4

Edges vs. the cut

(B, C)	1	✓ in T
(A, B)	2	✓ in T
(A, C)	3	inside S
(B, D)	4	✓ added
(C, D)	5	inside S
(C, E)	6	crosses
(D, E)	7	crosses

sorted by weight

Priority queue Q

v	key	π	
A	0	–	in S
B	2	A	in S
C	1	B	in S
D	4	B	in S
E	6	C	in Q

Q = {E:6}

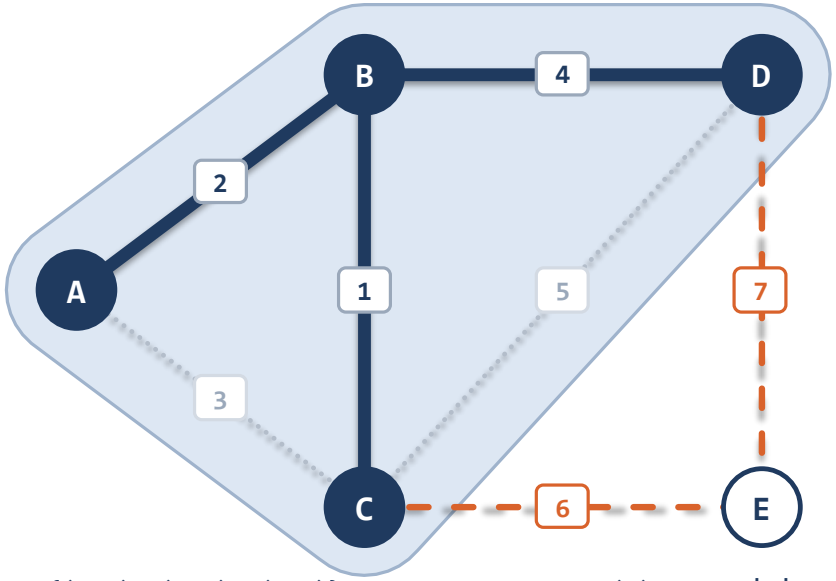
key[v] = lightest edge from v into S

EXTRACT-MIN returns D (key 4). Add (B, D) to T. w(T) = 7.

Update neighbours still in Q: key[E] stays 6, since w(D, E) = 7 is not smaller.
(A, C) weighs only 3, but both ends are in S: it does not cross the cut, and adding it closes a cycle.

Prim trace, step 4 of 4: which edge does Prim add?

S shaded · crossing edges dashed · T in bold



T = {(A,B), (B,C), (B,D)} w(T) = 7 |T| = 3 / 4

Edges vs. the cut

(B, C)	1	✓ in T
(A, B)	2	✓ in T
(A, C)	3	inside S
(B, D)	4	✓ in T
(C, D)	5	inside S
(C, E)	6	crosses
(D, E)	7	crosses

sorted by weight

Priority queue Q

v	key	π	
A	0	–	in S
B	2	A	in S
C	1	B	in S
D	4	B	in S
E	6	C	in Q

Q = {E:6}

key[v] = lightest edge from v into S

Cut (S, V − S) with S = {A, B, C, D}. Which edge does Prim add next?

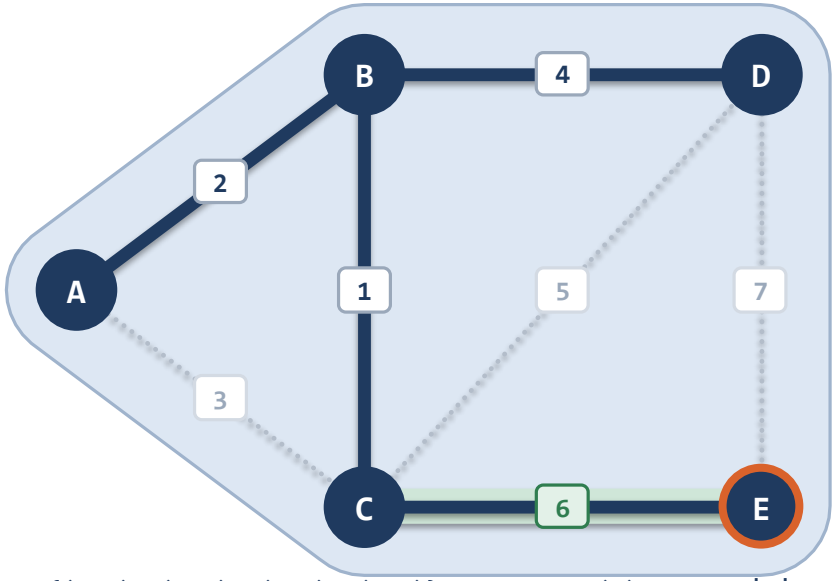
Crossing edges: (C, E) 6, (D, E) 7.

Vote by clicking an edge on the right.

(C, D) 5 | (C, E) 6 | (D, E) 7

Prim trace, step 4 of 4: add (C, E), w = 6

S shaded · crossing edges dashed · T in bold



T = {(A,B), (B,C), (B,D), (C,E)} w(T) = 13 |T| = 4 / 4

Edges vs. the cut

(B, C)	1	✓ in T
(A, B)	2	✓ in T
(A, C)	3	inside S
(B, D)	4	✓ in T
(C, D)	5	inside S
(C, E)	6	✓ added
(D, E)	7	inside S

sorted by weight

Priority queue Q

v	key	π	
A	0	–	in S
B	2	A	in S
C	1	B	in S
D	4	B	in S
E	6	C	in S

Q = ∅

key[v] = lightest edge from v into S

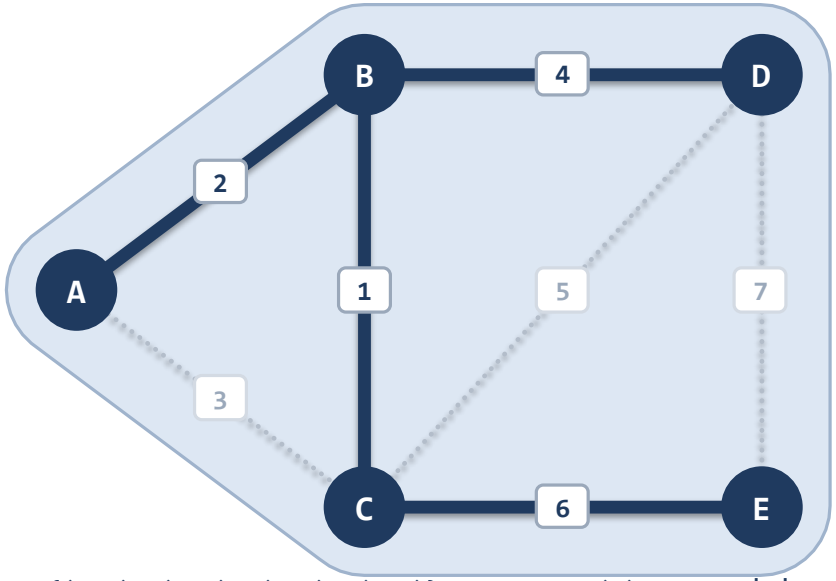
EXTRACT-MIN returns E (key 6). Add (C, E) to T. w(T) = 13.

All of E's neighbours are already in S, so no key changes.

(C, D) weighs only 5, but both ends are in S: it does not cross the cut, and adding it closes a cycle.

Prim trace: finished

S shaded · crossing edges dashed · T in bold



T = {(A,B), (B,C), (B,D), (C,E)} w(T) = 13 |T| = 4 / 4

Edges vs. the cut

(B, C)	1	✓ in T
(A, B)	2	✓ in T
(A, C)	3	not in T
(B, D)	4	✓ in T
(C, D)	5	not in T
(C, E)	6	✓ in T
(D, E)	7	not in T

sorted by weight

Priority queue Q

v	key	π	
A	0	–	in S
B	2	A	in S
C	1	B	in S
D	4	B	in S
E	6	C	in S

Q = ∅

key[v] = lightest edge from v into S

Q is empty, so Prim stops. T has 4 = |V| − 1 edges and w(T) = 2 + 1 + 4 + 6 = 13.

Each edge was the lightest one crossing the cut made by the tree at that moment.
The π pointers describe the tree: π[B] = A, π[C] = B, π[D] = B, π[E] = C.

Recap: one cut per step

Step	Tree side S	Edges crossing (S, V − S)	Light edge added
1	{A}	A–B 2 A–C 3	A–B (2)
2	{A, B}	B–C 1 A–C 3 B–D 4	B–C (1)
3	{A, B, C}	B–D 4 C–D 5 C–E 6	B–D (4)
4	{A, B, C, D}	C–E 6 D–E 7	C–E (6)

MST = {A–B, B–C, B–D, C–E}, total weight 13.

Step 3 is the one to remember: A–C (3) is lighter than B–D (4), but A and C are both in S, so A–C does not cross the cut. Adding it would close the cycle A – B – C. Prim never looks at it.

Question 1 of 4

If w(A, C) were 1.5 instead of 3, which edge would Prim add first?

- (A, B)
- (A, C)
- (B, C)

Click an answer.

Question 2 of 4

At step 3, why does Prim skip (A, C) even though 3 < 4?

- both ends in S
- weight too big
- A–C not in Q

Click an answer.

Question 3 of 4

What does key[v] hold for a vertex v that is still in Q?

- `lightest edge to S`
- `distance from A`
- `degree of v`

Click an answer.

Question 4 of 4

If Prim starts at E instead of A, is the MST different?

yes

no

Click an answer.

Time Complexity Analysis

Total Runtime = (Time to Build Heap)

+ (V * Time for Extract-Min)

+ (E * Time for Decrease-Key)

Time Complexity Analysis

Using a Binary Heap (Standard Implementation):

- Building initial Min-Heap: O(V)
- |V| Extract-Min operations: |V| * O(log V) = O(V log V)
- |E| Decrease-Key operations: |E| * O(log V) = O(E log V)
- Total Binary Heap Runtime: O(V log V + E log V) = O(E log V) on connected graphs (where E >= V - 1).

"bubble up" (percolate up) along the height of a complete binary tree.

Time Complexity Analysis

Using a Fibonacci Heap (Advanced Implementation):

- Extract-Min takes O(log V) amortized time.
- Decrease-Key takes O(1) amortized time.
- **Total Fibonacci Heap Runtime:** O(E + V log V)

Kruskal vs. Prim: When to Use Which?

Feature	Kruskal's Algorithm	Prim's Algorithm
Key Intuition	Global Forest Merging	Single Growing Tree
Primary Data Structure	Union-Find (DSU) + Sorted Edge Array	Min-Priority Queue (Min-Heap)
Binary Heap Time	O(E log V)	O(E log V)
Fibonacci Heap Time	N/A	O(E + V log V)
Best Graph Type	Sparse Graphs (E close to V) or pre-sorted edge lists	Dense Graphs (E close to V^2)

When E is approximately V^2

Kruskal's Algorithm on a Dense Graph

- Kruskal must sort **all** edges up front.
- Cost = O(E log E)
- On a dense graph where E = V^2:

Runtime = O(V^2 * log(V^2)) = O(2 * V^2 * log V) = O(V^2 log V)

Prim's Algorithm (Adjacency Matrix / Simple Array Implementation)

- If you implement Prim's using a simple **unsorted array** instead of a heap:
  - Extract-Min takes O(V) time, executed V times →O(V^2)
  - Decrease-Key / updating distances takes O(1) time, executed E times →O(E)
- On a dense graph where E = V^2:

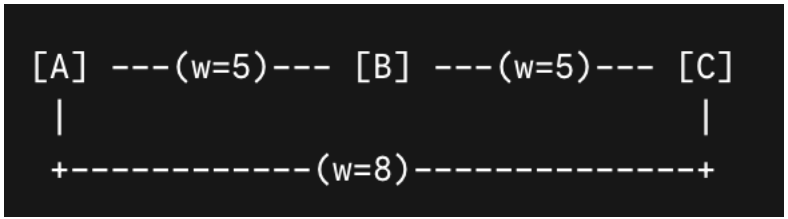
Runtime = O(V^2 + E) = O(V^2 + V^2) = O(V^2)

The Result: O(V^2) is strictly faster than O(V^2 log V) by a factor of log V.

Single-Source Shortest Paths (SSSP) & Dijkstra's Algorithm

Minimum Spanning Trees vs. Shortest Path Trees

- **MST Goal:** Minimize the **sum of all edge weights** connecting all vertices (Layout cost).
  - MST for graph above: Edges (A, B) and (B, C) with Total Weight = 10. (Path A -> C costs 10).
- **SSSP Goal:** Minimize the **path distance from a single source node s** to every other node v.
  - Shortest path A -> C: Direct edge (A, C) with Path Distance = 8.
- **Key Difference:** MST is a global structure tree; SSSP is root-specific.



Dijkstra's Algorithm: Edge Relaxation

- **dist[v] Definition:** The currently known shortest cumulativ

  path distance from source s to vertex v.
- **Path Relaxation Invariant:** If traveling to v via u yields a

  shorter path than v's current best known distance

```
PROCEDURE Relax(u, v, w):
    IF dist[u] + w(u, v) < dist[v]:
        dist[v] = dist[u] + w(u, v)
        parent[v] = u
        Decrease-Key(Q, v, dist[v])
```

dist[u] + w(u, v) < dist[v] => we update dist[v] = dist[u] + w(u, v)

```
ALGORITHM Dijkstra(G = (V, E), w, s):
    FOR EACH vertex v in V:
        dist[v] = Infinity
        parent[v] = NULL
        visited[v] = FALSE

    dist[s] = 0
    Build Min-Priority Queue Q containing all vertices in V keyed by dist[]

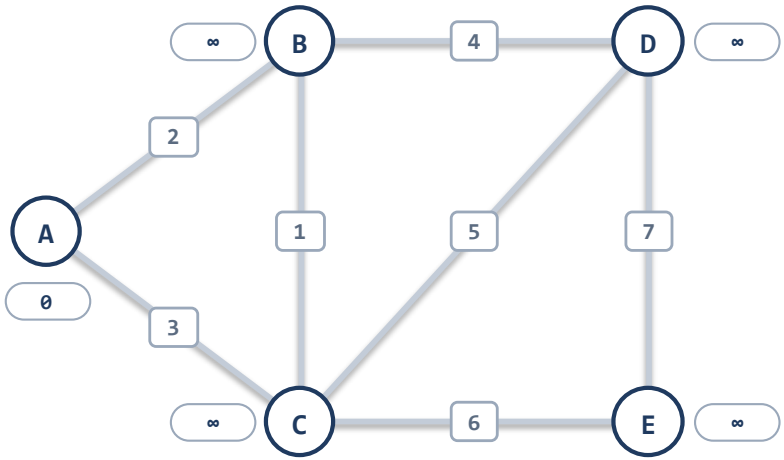
    WHILE Q is not empty:
        u = Extract-Min(Q)
        visited[u] = TRUE

        FOR EACH neighbor v of u:
            IF visited[v] == FALSE:
                Relax(u, v, w)
```

Feature	Prim's Algorithm (MST)	Dijkstra's Algorithm (SSSP)
Priority Key Definition	dist[v] = min edge weight w(u, v)	dist[v] = total path weight dist[u] + w(u, v)
Data Structure Engine	Min-Priority Queue (Extract-Min, Decrease-Key)	Min-Priority Queue (Extract-Min, Decrease-Key)
Greedy Choice	Pick node closest to the growing tree	Pick node closest to the source vertex

Dijkstra trace: initial state

S filled · final tree solid · tentative π dashed



Where we are in the code

```
DIJKSTRA(G, w, s)
1  INIT-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V   // key: v.d
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

Estimates d and parents π

v	d	π	
A	0	–	in Q
B	∞	–	in Q
C	∞	–	in Q
D	∞	–	in Q
E	∞	–	in Q

Priority queue Q (min first)

A:0 | B:∞ | C:∞ | D:∞ | E:∞

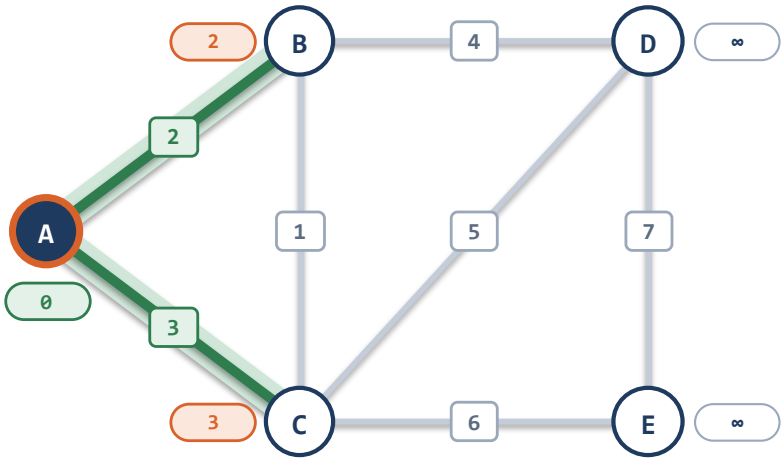
next EXTRACT-MIN → A

Initialize: d[A] = 0, every other d = ∞, every π = NIL. S = ∅ and Q holds all five vertices.

d[v] is an upper bound on the true distance δ(A, v). Relaxation can only lower it.
Click → to run the first EXTRACT-MIN. It must return A, the only finite estimate.

Step 1 of 5: extract A, d[A] = 0

S filled · final tree solid · tentative π dashed



RELAX each edge out of A

(A, B) ✓ update

`0 + 2 = 2 < ∞:  d[B] ← 2, π[B] ← A`

(A, C) ✓ update

`0 + 3 = 3 < ∞:  d[C] ← 3, π[C] ← A`

Estimates d and parents π

v	d	π	
A	0	–	in S (final)
B	2	A	in Q
C	3	A	in Q
D	∞	–	in Q
E	∞	–	in Q

Priority queue Q (min first)

B:2 | C:3 | D:∞ | E:∞

next EXTRACT-MIN → B

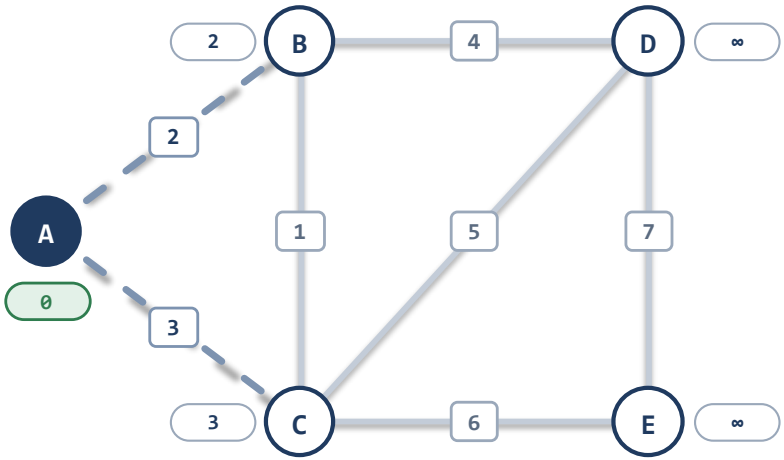
EXTRACT-MIN returns A: d[A] = 0 is the smallest estimate in Q, so it is final.

Relax: B: ∞ → 2; C: ∞ → 3.

The two estimates in Q that are finite now are path lengths of one edge each.

Step 2 of 5: which vertex comes out of Q?

S filled · final tree solid · tentative π dashed



Where we are in the code

```
DIJKSTRA(G, w, s)
1  INIT-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V   // key: v.d
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

Estimates d and parents π

v	d	π	
A	0	–	in S (final)
B	2	A	in Q
C	3	A	in Q
D	∞	–	in Q
E	∞	–	in Q

Priority queue Q (min first)

B:2 | C:3 | D:∞ | E:∞

next EXTRACT-MIN → B

Step 2: which vertex does EXTRACT-MIN return?

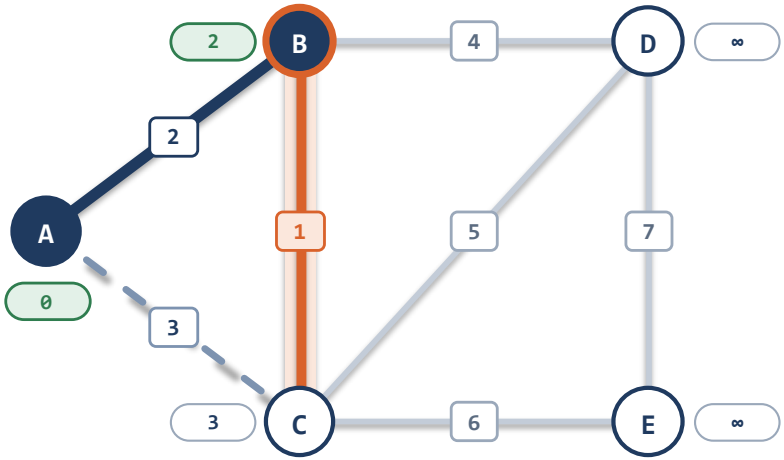
Q holds B:2 C:3 D:∞ E:∞. S = {A}.

Vote by clicking a vertex on the right.

B d = 2 | C d = 3 | D d = ∞ | E d = ∞

Step 2 of 5: extract B. Predict one relaxation

S filled · final tree solid · tentative π dashed



RELAX each edge out of B

(B, A) — already in S

A is final: d[A] = 0

(B, C) — ?

2 + 1 = 3 vs d[C] = 3. < ?

(B, D) — pending

2 + 4 = 6 vs d[D] = ∞

Estimates d and parents π

v	d	π	
A	0	–	in S (final)
B	2	A	in S (final)
C	3	A	in Q
D	∞	–	in Q
E	∞	–	in Q

Priority queue Q (min first)

C:3 | D:∞ | E:∞

EXTRACT-MIN returned B (d = 2). Now relax (B, C): 2 + 1 = 3.

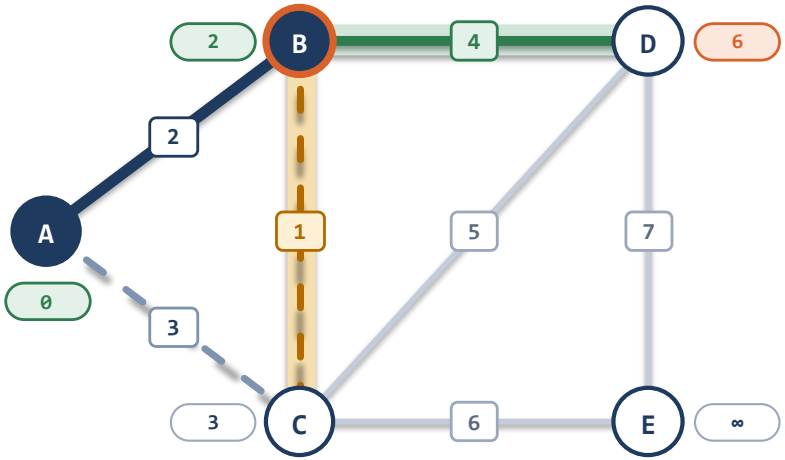
That matches d[C] = 3 exactly. Does π[C] change from A to B?
Vote on the right.

Yes: π[C] ← B

No: π[C] stays A

Step 2 of 5: extract B, d[B] = 2

S filled · final tree solid · tentative π dashed



RELAX each edge out of B

(B, A) — already in S

A is final: d[A] = 0

(B, C) — = tie: no change

2 + 1 = 3, not < 3: π[C] stays A

(B, D) — ✓ update

2 + 4 = 6 < ∞: d[D] ← 6, π[D] ← B

Estimates d and parents π

v	d	π	
A	0	–	in S (final)
B	2	A	in S (final)
C	3	A	in Q
D	6	B	in Q
E	∞	–	in Q

Priority queue Q (min first)

C:3 | D:6 | E:∞

next EXTRACT-MIN → C

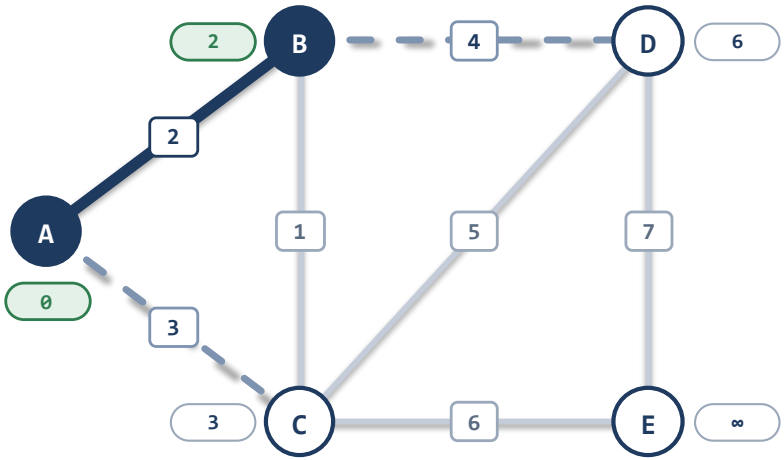
EXTRACT-MIN returns B: d[B] = 2 is the smallest estimate in Q, so it is final.

Relax: C: 3 = 3, a tie, so π[C] stays A; D: ∞ → 6.

The tie at C: RELAX tests d[v] > d[u] + w strictly, so an equal path leaves π alone.

Step 3 of 5: which vertex comes out of Q?

S filled · final tree solid · tentative π dashed



Where we are in the code

```
DIJKSTRA(G, w, s)
1  INIT-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V   // key: v.d
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

Estimates d and parents π

v	d	π	
A	0	–	in S (final)
B	2	A	in S (final)
C	3	A	in Q
D	6	B	in Q
E	∞	–	in Q

Priority queue Q (min first)

C:3 | D:6 | E:∞

next EXTRACT-MIN → C

Step 3: which vertex does EXTRACT-MIN return?

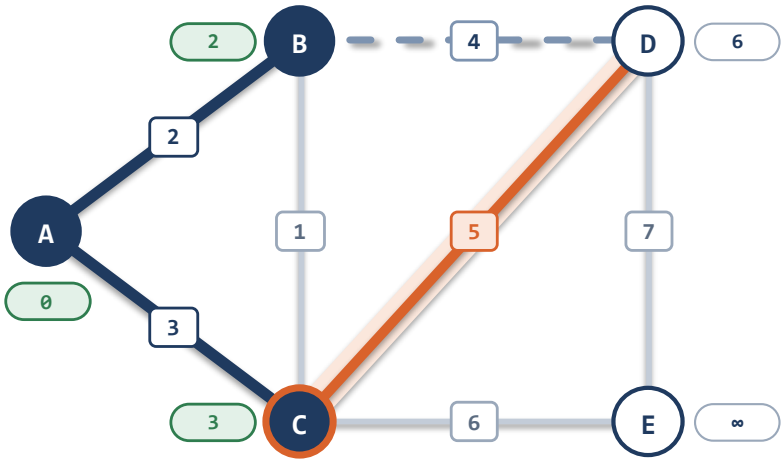
Q holds C:3 D:6 E:∞. S = {A, B}.

Vote by clicking a vertex on the right.

C d = 3 | D d = 6 | E d = ∞

Step 3 of 5: extract C. Predict one relaxation

S filled · final tree solid · tentative π dashed



RELAX each edge out of C

(C, A) — already in S

A is final: d[A] = 0

(C, B) — already in S

B is final: d[B] = 2

(C, D) — ?

3 + 5 = 8 vs d[D] = 6. < ?

(C, E) — pending

3 + 6 = 9 vs d[E] = ∞

Estimates d and parents π

v	d	π	
A	0	–	in S (final)
B	2	A	in S (final)
C	3	A	in S (final)
D	6	B	in Q
E	∞	–	in Q

Priority queue Q (min first)

D:6 | E:∞

EXTRACT-MIN returned C (d = 3). Now relax (C, D): 3 + 5 = 8.

Currently d[D] = 6. Does d[D] change?

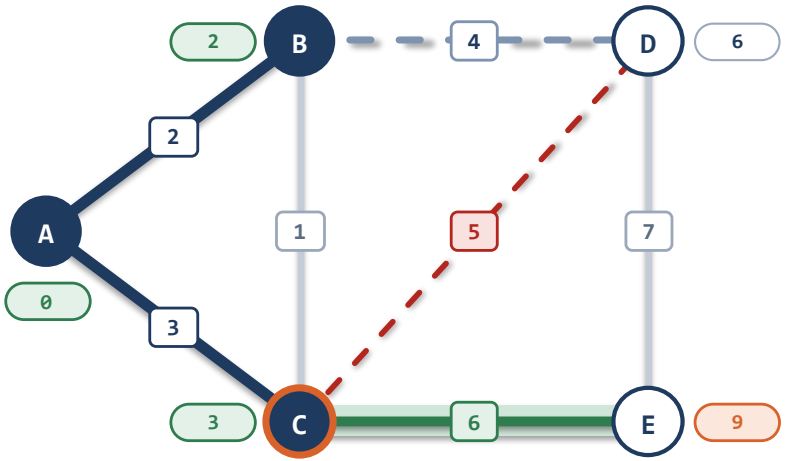
Vote on the right.

Yes: d[D] ← 8

No: d[D] stays 6

Step 3 of 5: extract C, d[C] = 3

S filled · final tree solid · tentative π dashed



RELAX each edge out of C

(C, A) — already in S

A is final: d[A] = 0

(C, B) — already in S

B is final: d[B] = 2

(C, D) — ✗ no change

3 + 5 = 8 > 6: d[D] stays 6

(C, E) — ✓ update

3 + 6 = 9 < ∞: d[E] ← 9, π[E] ← C

Estimates d and parents π

v	d	π	
A	0	–	in S (final)
B	2	A	in S (final)
C	3	A	in S (final)
D	6	B	in Q
E	9	C	in Q

Priority queue Q (min first)

D:6 | E:9

next EXTRACT-MIN → D

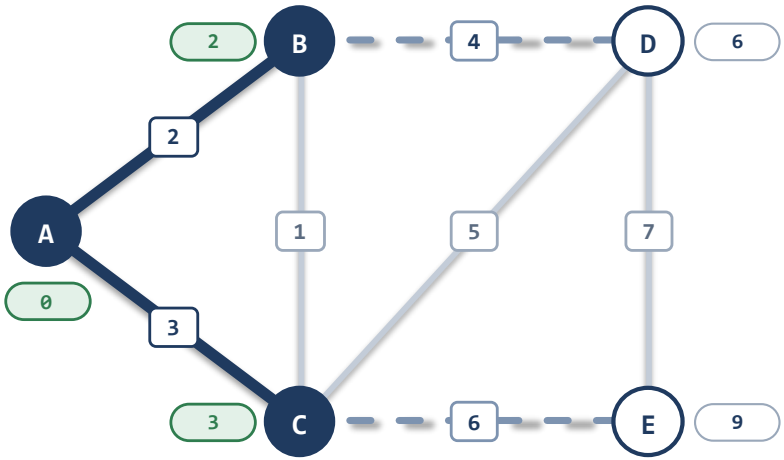
EXTRACT-MIN returns C: d[C] = 3 is the smallest estimate in Q, so it is final.

Relax: D: 8 > 6, no change; E: ∞ → 9.

Why doesn't D update? The path through C costs 8, but the path through B already costs 6.

Step 4 of 5: which vertex comes out of Q?

S filled · final tree solid · tentative π dashed



Where we are in the code

```
DIJKSTRA(G, w, s)
1  INIT-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V   // key: v.d
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

Estimates d and parents π

v	d	π	
A	0	–	in S (final)
B	2	A	in S (final)
C	3	A	in S (final)
D	6	B	in Q
E	9	C	in Q

Priority queue Q (min first)

D:6 | E:9

next EXTRACT-MIN → D

Step 4: which vertex does EXTRACT-MIN return?

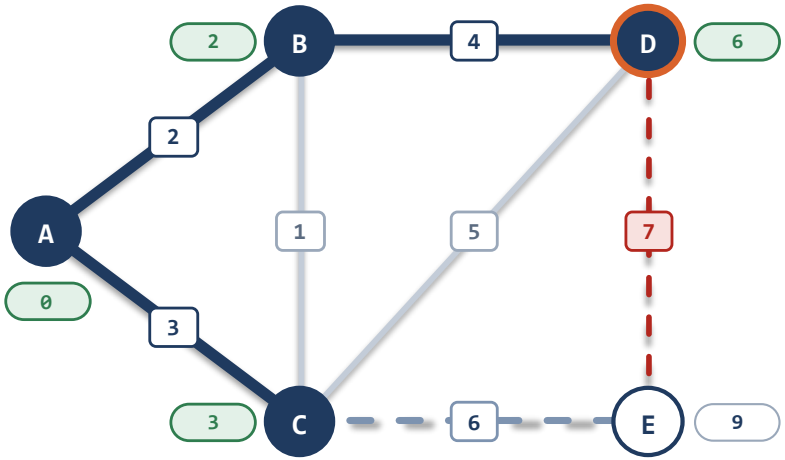
Q holds D:6 E:9. S = {A, B, C}.

Vote by clicking a vertex on the right.

D d = 6 | E d = 9

Step 4 of 5: extract D, d[D] = 6

S filled · final tree solid · tentative π dashed



RELAX each edge out of D

(D, B) — already in S

B is final: d[B] = 2

(D, C) — already in S

C is final: d[C] = 3

(D, E) — ✗ no change

6 + 7 = 13 > 9: d[E] stays 9

Estimates d and parents π

v	d	π	
A	0	–	in S (final)
B	2	A	in S (final)
C	3	A	in S (final)
D	6	B	in S (final)
E	9	C	in Q

Priority queue Q (min first)

E:9

next EXTRACT-MIN → E

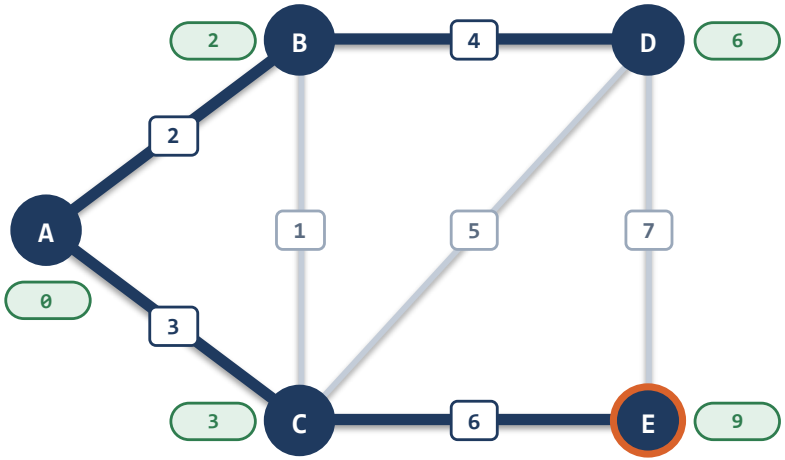
EXTRACT-MIN returns D: d[D] = 6 is the smallest estimate in Q, so it is final.

Relax: E: 13 > 9, no change.

A failed relaxation: going on through D to E costs 13, worse than the 9 we have.

Step 5 of 5: extract E, d[E] = 9

S filled · final tree solid · tentative π dashed



RELAX each edge out of E

(E, C) — already in S

C is final: d[C] = 3

(E, D) — already in S

D is final: d[D] = 6

Estimates d and parents π

v	d	π	
A	0	–	in S (final)
B	2	A	in S (final)
C	3	A	in S (final)
D	6	B	in S (final)
E	9	C	in S (final)

Priority queue Q (min first)

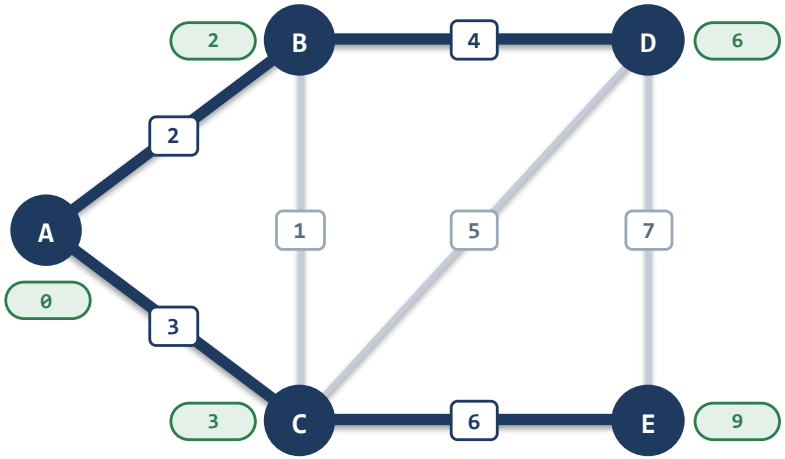
Q = ∅

Q = {E}, so EXTRACT-MIN returns E. d[E] = 9 is now final.

Every neighbour of E is already in S. Those relaxations cannot succeed.
E was the last vertex in Q. Its estimate, 9, is final.

Dijkstra trace: finished

S filled · final tree solid · tentative π dashed



Where we are in the code

```
DIJKSTRA(G, w, s)
1  INIT-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V   // key: v.d
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

Estimates d and parents π

v	d	π	
A	0	–	in S (final)
B	2	A	in S (final)
C	3	A	in S (final)
D	6	B	in S (final)
E	9	C	in S (final)

Priority queue Q (min first)

Q = ∅

Q is empty, so Dijkstra stops. Every d[v] now equals the shortest-path distance δ(A, v).

B: A → B = 2, C: A → C = 3, D: A → B → D = 6, E: A → C → E = 9.

The π pointers form the shortest-path tree A–B, A–C, B–D, C–E (total edge weight 15).

Recap: one EXTRACT-MIN per step

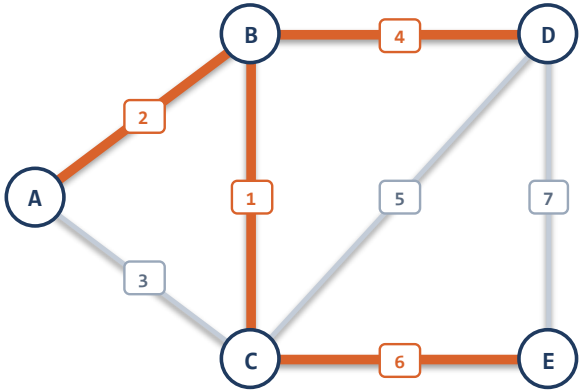
Step	Extract	Relaxations (edges back into S omitted)	d[A, B, C, D, E] after	π set
1	A, 0	B: ∞→2 C: ∞→3	0, 2, 3, ∞, ∞	B←A, C←A
2	B, 2	C: 3 = 3 tie D: ∞→6	0, 2, 3, 6, ∞	D←B
3	C, 3	D: 8 > 6 ✗ E: ∞→9	0, 2, 3, 6, 9	E←C
4	D, 6	E: 13 > 9 ✗	0, 2, 3, 6, 9	–
5	E, 9	none	0, 2, 3, 6, 9	–

Distances: B = 2, C = 3, D = 6, E = 9. Tree: A–B, A–C, B–D, C–E.

Green = successful relaxation, red = failed, amber = tie. Steps 3 and 4 show that a finite estimate is only replaced by a strictly shorter one; the tie at step 2 keeps π[C] = A.

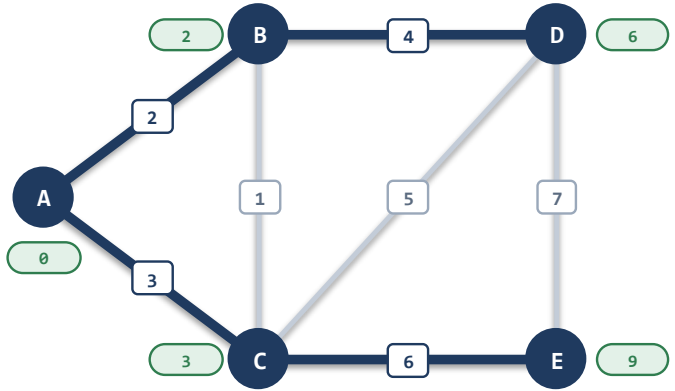
Prim vs. Dijkstra on the same graph

Prim: minimum spanning tree



A–B, B–C, B–D, C–E total 13

Dijkstra from A: shortest-path tree



A–B, A–C, B–D, C–E total 15

Only C differs. Prim adds B–C because 1 is the lightest edge crossing the cut.

Dijkstra keeps A–C: the route through B costs 2 + 1 = 3, which is not < 3. Key update: Prim v.key = w, Dijkstra v.d = u.d + w.

Careful: here the MST is also a shortest-path tree (it ties at C). Make w(A, C) = 2.5 and it no longer is: A→B→C = 3 > 2.5.

Running time

Q implemented as	EXTRACT-MIN (×V)	DECREASE-KEY (×E)	Total
Binary min-heap	$O(\log V)$	$O(\log V)$	$\mathbf{O((V + E) \log V)}$
Fibonacci heap	$O(\log V)$ amort.	$O(1)$ amort.	$\mathbf{O(E + V \log V)}$
Unsorted array	$O(V)$	$O(1)$	$\mathbf{O(V^2)}$

Each vertex is extracted once: |V| EXTRACT-MINs.
Each adjacency entry is relaxed once: 2|E| RELAX calls (undirected), at most that many DECREASE-KEYs.
Same costs as Prim. Weights must be ≥ 0; with negative edges use Bellman–Ford, O(VE).

In our example

EXTRACT-MIN calls	**5**
RELAX calls	**14**
successful	**4**
failed (d too big)	**2**
ties	**1**
into S (final)	**7**

Question 1 of 4

If w(B, C) were 0.5 instead of 1, what would π[C] be at the end?

A

B

D

Click an answer.

Question 2 of 4

Why does Dijkstra require w(u, v) ≥ 0?

- `final d could drop`
- `heap rejects negatives`
- `RELAX would loop`

Click an answer.

Question 3 of 4

Running time with a binary min-heap?

- $O((V + E) \log V)$
- $O(V^2)$
- $O(E + V \log V)$

Click an answer.

Question 4 of 4

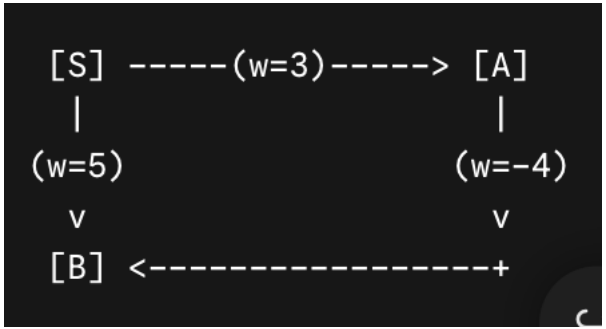
Is a minimum spanning tree always a shortest-path tree?

yes

no

Click an answer.

Why Dijkstra Fails on Negative Edge Weights



- **Execution Failure Step:**
  - Dijkstra extracts A (dist[A] = 3) and B (dist[B] = 5).
  - Because dist[A] < dist[B], node A is extracted first and marked **Finalized / Visited**.
  - Dijkstra assumes that because all edge weights are positive, no future path can make A shorter than 3.
  - However, path S -> A -> B has total weight 3 + (-4) = -1, which is shorter than dist[B] = 5.
  - Because B was already finalized, Dijkstra fails to compute the correct shortest path to B.
- **Takeaway:** Dijkstra requires **all edge weights to be non-negative** (w(e) >= 0). For negative weights, we must use the **Bellman-Ford algorithm**.